

```

---
title: "Algoritmos"
author: "Rocio Anabel Negroni"
date: "2026-04-01"
output:
  pdf_document: default
  html_document: default
  word_document: default
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```

```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <<http://rmarkdown.rstudio.com>>.

When you click the *Knit* button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a matlab y trabaja generalmente con variables que pueden ser matrices. Para escribir un código oprimimos la tecla +C en verde y R.

```

```{r pressure}
plot(pressure,main="Presión del gas ideal",ylab="Presión
[hPa]",xlab="Temperatura [K]",type="b", col="blue")
```

```

```

```{r}
a=38
a=40
b=60
c=b-a
c
```

```

```

```{r}
Y=40
alpha<-30
```

```

Uso de data sets o base de datos internos de R

Usando el comando data en la consola obtenemos un listado de distintos datos ya cargados en la base de datos de R. Estos datos están dados en tablas/matrices de las cuales podemos elegir columnas específicas si escribimos signo pesos después del comando de información específico. Por ejemplo cars\$speed. Nos saltan todas la velocidades que antes la encontrabamos en forma de columna en la matriz de cars.

Conclusion

El uso de Posit Cloud representa una solución práctica y accesible para iniciarse en el análisis de datos con R, especialmente en entornos académicos y colaborativos. Su facilidad de acceso, sin necesidad de instalaciones complejas, favorece el aprendizaje y el trabajo en equipo desde cualquier lugar. No obstante, al depender de internet y contar con recursos limitados, puede no ser la opción más adecuada para proyectos de gran escala o que requieran alto rendimiento computacional. En consecuencia, se posiciona como una herramienta ideal para la formación, la experimentación y el desarrollo de proyectos intermedios, mientras que los trabajos más exigentes pueden beneficiarse de un entorno local más robusto.

Tabla comparativa: Posit Cloud vs R local

| Característica | Posit Cloud | R local (instalado) |
|----------------|-------------|---------------------|
|----------------|-------------|---------------------|

| en PC) | | |
|----------------------------------|---|----------------|
| Instalación | No requiere instalación | Requiere |
| instalación de R y RStudio | | |
| Acceso | Desde cualquier dispositivo con internet | Solo desde la |
| computadora instalada | | |
| Conexión a internet | Obligatoria | No obligatoria |
| | | |
| Recursos computacionales | Limitados (según el plan) | Dependen del |
| hardware de la PC | | |
| Rendimiento | Adecuado para proyectos pequeños/medianos | Mejor para |
| proyectos grandes o pesados | | |
| Trabajo colaborativo | Muy sencillo (compartir proyectos online) | Requiere |
| herramientas externas (Git) | | |
| Actualizaciones | Automáticas | Manuales |
| | | |
| Ideal para | Aprendizaje y prototipado | Desarrollo |
| profesional y análisis intensivo | | |

Data set en R

```
**¿Qué es un dataset?**
```

Es una colección organizada de información, generalmente estructurada en forma de tabla para poder analizarla fácilmente.

****Tipos de datasets en R:****

****Internos**** → vienen incluidos en R o paquetes

```
Ej: `iris`, `mtcars`
```

****Externos**** → archivos que importás

Ej: CSV, Excel, TXT

Datasets internos

-Ya están disponibles sin descargar

- Se usan para aprender y practicar

****Comandos básicos:****

 $\{r\}$

```
data()      # lista todos los datasets
```

```
data(iris) # carga uno
```

Datasets externos

- Se cargan desde archivos

****Ejemplos:****

 $\vdots$

```
read.csv("archivo.csv")
```

```
read.table("archivo.txt")
```

Exploración de datos

 $\dots r$

```
head(datos)      # primeras filas
```

```
summary(datos) # resumen estadístico
```

```
str(datos)      # estructura
```

```
## Comando Plot
```

Es un comando que permite graficar cualquier fuente de datos que se le asigne. Se coloca plot y entre paréntesis lo que se quiera graficar. Sepando con comas se van agregando los cambios que se le quieran hacer al gráfico, como el título, lo que esta escrito sobre los ejes, etc. Si no sabemos como es el comando para cambiar algun parámetro del gráfico lo podemos buscar escribiendo en la consola signo de pregunta? seguido del comando plot y explicara en detalle el comando y sus variantes. En la consola podemos escribir y haciendo enter visualizar en la pestaña de la derecha como se vería. Si quedó acorde a lo que queríamos lo copiamos y pegamos en el pestaña de la izquierda superior Source, aquí es donde se hacen todos los cambios que se verán reflejados en el documento que estamos haciendo. Lo que se haga en la consola no queda

plasmado.

Descarga de documentos

Tenemos distintas opciones para tejer el documento, para usarlas hay que oprimir el triángulo del ovillo de lana *Knit* y veremos como se despliegan distintas opciones: Knit to HTML nos dará una página web, Knit to PDF nos permitirá descargar un PDF, Knit to WORD nos permitirá descargar un archivo WORD.

Funciones estadísticas

Para utilizar funciones estadísticas debemos colocar un código, por lo tanto, debemos oprimir +C y luego R, se nos despliega el espacio para escribir el código. Allí escribimos el comando `rnorm`

```
```{r}
z1<-rnorm(351,22,5)
z1
```
```

```
```{r}
plot(z1)
```
```

hist es un comando para hacer un histograma.

```
```{r}
hist(z1, main="Histograma de edades",breaks=50)
```
```

density es un comando para hacer un gráfico similar a una campana de Gauss pero no es perfecto.

```
```{r}
density(z1)
```
```

```
```{r}
plot(density(z1),type="p",col="purple")
lines(density(z1),col="yellow")
```
```

Normal {stats} R Documentation

The Normal Distribution

Description

Density, distribution function, quantile function and random generation for the normal distribution with mean equal to mean and standard deviation equal to sd.

Usage

```
dnorm(x, mean = 0, sd = 1, log = FALSE)
pnorm(q, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)
qnorm(p, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)
rnorm(n, mean = 0, sd = 1)
```

Arguments

x, q
vector of quantiles.
p vector of probabilities.
n
number of observations. If `length(n) > 1`, the length is taken to be the number required.

```
```{r}
plot(z1)
w1<-length(z1)
w1
x1<-(500:850)
x1
plot(x1, z1, col="darkblue")
lines(x1, z1, type="l", col="orange")
```
```

```
...
```

```
# Clase 8 de Abril
```

```
## Ejercicio 1 de algoritmos
```

```
**Consigna 1:** Las últimas 3 cifras de mi DNI son 067. Crear una variable que tenga ese número.
```

```
```{r}
DNI=067
```
```

```
**Consigna 2:** Crear un vector que tenga todos los números desde el 1 hasta el 067.
```

```
```{r}
for(i in 1:067)
print(1:i)
```
```

```
```{r}
secuencia_DNI <- c()
for(i in 1:067){
secuencia_DNI <- c(secuencia_DNI, i)
}
```
```

```
## Ejercicio 2
```

```
**Consigna 3:** Calcular la suma de todos los valores del vector secuencia_DNI
```

```
```{r}
Total<-0
Valor_final<-length(secuencia_DNI)
for(i in 1:Valor_final){
 Total<-Total+i
}
Total
print(Total)
```
```

```
## Ejercicio 3
```

```
**Consigna 4:** Repetir el ejercicio anterior pero en Phyton
```

```
{python}
secuenciaDNI = range(1, 068) # incluye 067
total = sum(secuenciaDNI)
print(total)
```

```
## Ejercicio 4
```

```
**Consigna 5:** ¿Cuánto tarda en correr el *Ejercicio 3*?
```

```
```{r}
system.time(for(i in 1:100) mad(runif(1000)))
```
```

El comando system.time sirve para conocer cuantos milisegundo tarda en ejecutarse un bloque de código

```
## Ejercicio 5
```

```
Determinar CPU time used
```

```
```{r}
```

```

require(stats)
system.time(for(i in 1:100) mad(runif(1000)))
```


```

```{r}
require(stats)
DNI= 67
secuencia= 1:DNI

total <- 0
Valor_final <- length(secuencia)

for (i in 1:Valor_final){
  total <- total + secuencia[i]
}
print(total)
```


```

```{r}
sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time- start_time
```

```


```


```

Ejercicio 6: **Comparación de performance con system.time()**

```

```{r}
system.time({
A <- numeric(50000)
for (i in 1:50000) {
A[i] <- i * 2
}
})
system.time({
B <- seq(from = 2, to = 100000, by = 2)
})
```

```

Biblioteca tictoc

El uso de una biblioteca en R consiste en incorporar un conjunto de funciones adicionales que amplían las capacidades del lenguaje. Estas bibliotecas contienen procedimientos previamente desarrollados que permiten realizar tareas específicas de manera más sencilla y eficiente. Para poder utilizarlas, primero es necesario instalarlas mediante el comando `install.packages()`, lo que descarga el paquete al entorno de trabajo. Luego, se debe cargar con `library()`, paso que habilita sus funciones para ser utilizadas en la sesión actual.

En el caso de la biblioteca `tictoc`, su finalidad es medir el tiempo de ejecución de un bloque de código. Esta incorpora las funciones `tic()` y `toc()`, que actúan como un cronómetro: la primera inicia la medición y la segunda la finaliza mostrando el tiempo transcurrido. Estas funciones son similares a las utilizadas en Octave o Matlab, manteniendo una lógica de uso sencilla y familiar.

Si bien R dispone de herramientas propias para medir tiempos, como `system.time()`, la biblioteca `tictoc` ofrece una alternativa más cómoda y clara, especialmente cuando se desea evaluar y comparar el rendimiento de distintos procedimientos. De este modo, facilita el análisis de eficiencia del código y mejora la organización del trabajo.

```

```{r eval=FALSE}
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
[1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
[1] "...suena el despertador"
toc()
```

```

Biblioteca rbenchmark

La función `benchmark` del paquete `rbenchmark` se describe en su documentación como un contenedor sencillo alrededor de la función base `system.time()` de R. Esto significa que internamente

utiliza el mismo mecanismo para medir el tiempo de ejecución, pero lo presenta de una manera más práctica y organizada para el usuario.

Aunque `system.time()` permite medir cuánto tarda una expresión en ejecutarse, su uso repetido puede volverse poco cómodo cuando se desean comparar varias expresiones o realizar múltiples repeticiones de cada una. En cambio, `benchmark` simplifica este proceso, ya que permite evaluar varias expresiones dentro de una sola llamada, indicando además cuántas repeticiones se desean realizar.

Otra ventaja importante es que los resultados obtenidos se devuelven en forma de data frame, lo que facilita su lectura, comparación y posterior análisis. De este modo, la función no solo automatiza la medición de tiempos, sino que también organiza la información de manera clara y estructurada, aportando mayor comodidad y eficiencia al trabajo de evaluación de rendimiento en R.

Implementación de una serie Fibonacci o Fibonacci

En matemáticas, la sucesión de Fibonacci es una sucesión infinita de números naturales que comienza de la siguiente manera:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, ...

La característica principal de esta sucesión es que, a partir de los dos primeros términos (0 y 1), cada número se obtiene sumando los dos anteriores.

Los elementos que forman esta sucesión se denominan números de Fibonacci.

Esta sucesión fue introducida en Europa por Leonardo de Pisa, matemático italiano del siglo XIII, más conocido como Fibonacci, en su obra *Liber Abaci*. Aunque ya era conocida en matemáticas orientales, su difusión en Europa se debe a este autor.

La sucesión de Fibonacci posee numerosas aplicaciones en diversas áreas, como la matemática, la informática, la teoría de juegos e incluso en fenómenos naturales y modelos de crecimiento. Su importancia radica tanto en su sencillez como en las múltiples propiedades y relaciones que presenta dentro del estudio de los números.

****Consigna 8:**** ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 20 ? Además mostrar la serie de Fibonacci hasta el número obtenido anteriormente.

```
```{r}
Inicialización
a <- 0
b <- 1
serie <- c(a, b) # guardamos la serie
contador <- 2 # ya tenemos 2 términos
Generar hasta superar 20
while (b <= 20) {
 siguiente <- a + b
 a <- b
 b <- siguiente
 serie <- c(serie, b) # agregamos a la serie
 contador <- contador + 1
}
Resultados
cat("Cantidad de iteraciones:", contador, "\n")
cat("Primer número mayor a 20:", b, "\n")
cat("Serie de Fibonacci hasta ese número:\n")
print(serie)
```
```

****Consigna 9:**** Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método `microbenchmark` para una muestra de tamaño 20.000

```
```{r}
Instalar paquete
if (!require(microbenchmark)) {
 install.packages("microbenchmark")
 library(microbenchmark)
}
Generar muestra
set.seed(123)
n <- 20000
vector <- sample(1:100000, n, replace = TRUE)
```

```
#Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
 n <- length(x)
 for (i in 1:(n-1)) {
 for (j in 1:(n-i)) {
 if (x[j] > x[j+1]) {
 temp <- x[j]
 x[j] <- x[j+1]
 x[j+1] <- temp
 }
 }
 }
 return(x)
}
#Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
 burbuja = bubble_sort(vector),
 sort_R = sort(vector),
 times = 5
)
print(resultado)
```

```

La penitencia de Newton

Algunos historiadores relatan que Isaac Newton, durante su etapa escolar, tuvo un maestro sumamente estricto que exigía absoluta atención en clase. Ante cualquier distracción o falta de disciplina, imponía como castigo una tarea particularmente tediosa: calcular para el día siguiente la suma de todos los números desde 1 hasta un número elegido al azar.

A simple vista, esta consigna implicaba realizar largas sumas parciales, una por una, lo que podía demandar horas de trabajo. El profesor, precavido, ya tenía resueltos estos cálculos y los llevaba anotados en un cuaderno voluminoso, similar en utilidad a una tabla de logaritmos, que transportaba cuidadosamente en su portafolio.

La anécdota cobra relevancia porque, según la tradición, el joven Newton habría encontrado una forma más rápida y elegante de realizar esas sumas, descubriendo un patrón que permitía obtener el resultado sin necesidad de sumar término por término. Más allá de su veracidad histórica, el relato ilustra cómo la observación y el razonamiento pueden transformar una tarea repetitiva en un problema matemático con una solución general.

****Consigna 10:**** Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1×10^{10} y verifica cuál de los dos es más eficiente.

```
```{r}
#Algoritmo 1: suma con for (iterativo)
n <- 1010
suma1 <- 0
for (i in 1:n){
 suma1 <- suma1 + i
}
suma1
#Complejidad:O(n)(suma uno por uno)
#Algoritmo 2: fórmula matemática (Gauss)
n <- 1010
suma2 <- n * (n + 1) / 2
suma2
#Complejidad:O(1)(una sola operación)
#Comparación de eficiencia:medir tiempos con system.time():
Método 1
system.time({
 suma1 <- 0
 for (i in 1:1010){
 suma1 <- suma1 + i
 }
})
Método 2
system.time({
 suma2 <- 1010 * (1010 + 1) / 2
})
```

```

Se desarrollaron dos algoritmos para sumar desde 1 hasta 1×10^{10} . El método iterativo presenta

complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente

```
##Matriz Inversa
```{r}
##Definir la matriz
A <- matrix(c(4, 7,
 2, 6),
 nrow = 2,
 byrow = TRUE)
```

```
##Calcular la inversa
A_inv <- solve(A)
```

```
##Mostrar resultado
A_inv
```
```

```
## Investigar con los gráficos de violin (de la biblioteca micro como se comporta una
computadora usando métodos de ordenamiento "burbuja" y compararlo con el método sort nativo de
R .
```

```
## Medir la performance del método kmean de agrupamiento. Realizar una introducción teórica
antes de medir con gráfico de violin.
```

```
```{r}
library(microbenchmark)
library(ggplot2)
library(dplyr)
```

```
Implementación bubble sort
bubble_sort <- function(x) {
 n <- length(x)
 for (i in 1:(n-1)) {
 for (j in 1:(n-i)) {
 if (x[j] > x[j+1]) {
 temp <- x[j]
 x[j] <- x[j+1]
 x[j+1] <- temp
 }
 }
 }
 return(x)
}
```

```
Datos de prueba
set.seed(123)
vec <- sample(1:5000, 500)
```

```
Benchmark
bench <- microbenchmark(
 bubble = bubble_sort(vec),
 sortR = sort(vec),
 times = 30
)
```

```
Convertir a ms
df <- as.data.frame(bench) %>%
 mutate(time_ms = time / 1e6)
```

```
Gráfico de violín
ggplot(df, aes(x = expr, y = time_ms)) +
 geom_violin(trim = FALSE, fill = "lightblue", alpha = 0.6) +
 geom_boxplot(width = 0.1, outlier.shape = NA) +
 labs(title = "Bubble Sort vs sort()",
 x = "Método",
 y = "Tiempo (ms)") +
```

```

theme_minimal()

...
```{r}
library(microbenchmark)
library(ggplot2)
library(dplyr)

# Implementación bubble sort
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}

# Datos de prueba
set.seed(123)
vec <- sample(1:5000, 500)

# Benchmark
bench <- microbenchmark(
  bubble = bubble_sort(vec),
  sortR = sort(vec),
  times = 30
)

# Convertir a ms
df <- as.data.frame(bench) %>%
  mutate(time_ms = time / 1e6)

# Gráfico de violín
ggplot(df, aes(x = expr, y = time_ms)) +
  geom_violin(trim = FALSE, fill = "blue", alpha = 0.6) +
  geom_boxplot(width = 0.1, outlier.shape = NA) +
  labs(title = "Bubble Sort vs sort()",
       x = "Método",
       y = "Tiempo (ms)") +
  theme_minimal()

...

```